FedWatcher

An Agentic System for FOMC Tone Extraction and Policy-Rate Nowcasting

Johnny Magon Lorenzo Mottinelli Leonardo Palanca
Sebastian Pasz
Programming in Finance II, Spring 2026 – Università della Svizzera italiana

June 7, 2026

GitHub: https://github.com/LeoPalanca/FEDWatcher
Live dashboard: https://fedwatcher.ellep.it

---

1 Project Plan

1.1 Problem

The U.S. Federal Reserve communicates monetary-policy intent through periodic FOMC statements and minutes. Markets re-price aggressively around these releases, but the textual signal embedded in the documents is high-dimensional, qualitative, and partially redundant with what is already implied by Fed-funds futures. *FedWatcher* asks a single, narrow question: *given the latest FOMC text and a current snapshot of macro and rates data, what is the most likely outcome of the next FOMC meeting (cut / hold / hike, in basis-point buckets)?*

1.2 Solution at a Glance

FedWatcher is an *agentic* pipeline made of three named agents that read from and write to a single SQLite database:

MonitorAgent
Scrapes new FOMC statements and minutes from the Federal Reserve website (and a synthetic `FakeFed` mirror used for scraper tests) and inserts them into the `documents` table. Although this component does not use an LLM, we still refer to it as an agent to maintain consistency across the FedWatcher architecture.

AnalystAgent
Splits each document into rhetorical sections, asks an LLM to score the tone of each section on a structured rubric (overall tone, inflation assessment, labor-market assessment, forward guidance), and writes the result into `sentiment` and its variants (`sentiment2`, `sentiment3`, `sentiment_w`) used for ablations.

StrategistAgent
Applies a time smoothing on all the stored sentiments and joins the final sentiment row with macro features (CPI, core CPI, unemployment, 2y yield, policy rate) from FRED and produces a probability vector over the next-meeting outcome plus a short natural-language explanation.

A FastAPI service exposes the database read-only over HTTPS; a static HTML/JavaScript dashboard renders document feeds, macro charts and signal tables on top of that API.

1.3 Generic Project Criteria Coverage

The course allows any project that combines three of the listed elements with at least one *advanced* (starred) component. FedWatcher covers:

- **Cloud-based infrastructure**: deployed on a Debian VPS (`Bavaria`, OVH) behind Nginx; `systemd` supervises the API; `cron` drives all ingestion and analyst loops.
- **Large language model**: AnalystAgent uses an LLM (Anthropic Claude and OpenAI models via OpenRouter) for structured tone extraction; StrategistAgent uses an LLM for narrative synthesis.
- **Machine learning**: an ordered/multinomial nowcast over $\{-50, -25, 0, +25, +50\}$ basis-point buckets, trained on macro features and tone aggregates.
- **Non-trivial database**: SQLite with $\sim$15 related tables (documents, multi-version sentiment, signals, weights, section weight calibration, market and macro data, policy moves).
- **Real-time data**: hourly cron polling of Fed pages and FRED series.
- **Advanced data visualization**: an interactive single-page dashboard with document feed, modal viewer, range/scale pickers, and per-table charts.
- **Own API**: a read-only FastAPI service with health, table, snapshot, and document endpoints.

1.4 Repository Layout

- `agents/` – runtime agents (analyst variants, dual-model analyst, w_agent, strategist).
- `scripts/` – ingestion, backfill, schema, and cron-wrapper shell scripts.
- `app/` – FastAPI app exposing the SQLite store.
- `fedwatcher/` – static dashboard (HTML, CSS, vanilla JS).
- `fakefed/` – synthetic Fed-like static site used as a scraper fixture.
- `db/schema.sql` – the canonical schema (single source of truth).
- `AGENTS.md` – contributor contract for human and AI collaborators.

2 Project Diary

The full GitHub commit log is the canonical record of what was done and when; what follows is a curated narrative grouped into five phases, with representative commits referenced by short SHA so that the reader can drill into the diff online. Author initials are used for brevity: **LP** = Leonardo Palanca, **SP** = Sebastian Pasz, **LM** = Lorenzo Mottinelli, **JM** = Johnny Magon.

2.1 Phase 1 – Bootstrap (April 18, 2026)

The repository was created with a placeholder README (`c27c04b`, LP) and four initial pushes from each team member, mostly establishing folder layout, a Python virtual environment and dependency list (`1dacea2`, LP), and a first SQLite `test_db.py` (`9999da4`, LP). The first non-trivial component was a hidden-folder scraper exploration (`9a9def8`, "don't look here scrapers", LP) which fed into a `MonitorAgent` prototype the same evening (`39d344f`, JM). A first HTML dashboard mockup and an architecture sketch landed alongside (`128816b`, SP), followed by improvements to the monitor (`0a8731e`, JM) and a working `fetch_doc_text` helper (`8989d1a`, JM). The choice to use Anthropic-style commit messages and to keep the database URL secret (`80dbfb0`, LP) was made on day one.

2.2 Phase 2 – Architecture Consolidation (April 30, 2026)

After a pause to clarify the brief, the team rewrote the AGENTS contract and made it the entry point for any contributor or AI agent (`0abc90c`, LP). The README was consolidated and the architecture section moved into it (`299a76a`, LP). A synthetic Fed-like static site, `FakeFed`, was deployed behind Nginx as a reproducible scraper fixture (`b738e84`, LP), and the homepage of `fedwatcher.ellep.it` was published as a static landing dashboard (`1779a51`, LP). Dead files from the bootstrap phase were removed (`d1935b0`, `73c51ce`, LP).

2.3 Phase 3 – Analyst v1 and Macro Ingestion (May 5–13, 2026)

The first end-to-end `AnalystAgent` appeared on May 5: it segments each FOMC document into rhetorical sections (assessment, outlook, policy stance, vote) and asks an LLM to fill a structured rubric per section (`b8370d2`, SP). Documentation followed in lockstep (`a48a688`, `8f8fb75`, `2b38c26`, LP and SP). OpenRouter was added as a vendor-agnostic LLM gateway so the team could swap between Claude and OpenAI without code changes (`6ff8127`, LP). On May 7 the FRED backfill landed for CPI, core CPI, unemployment and 2y yields (commit grouped under several README updates by LP, see `4a1014e`). Mid-May the monitor was made source-aware so it can ingest both the live Fed site and `FakeFed` (`b209be1`, LP), and macro refresh was extended to multiple series (`f8bf297`, LP).

2.4 Phase 4 – Multi-Analyst, Strategist and Automation (May 19–22, 2026)

The team intentionally produced several competing analyst implementations to compare segmentation strategies, prompt styles and LLM backends. "Better analyst" and "better analyst 2" (`3b93a26`, `34c1243`, SP) explored alternative section weights, and a *dual-model* analyst was added that runs two LLMs in parallel and reconciles their outputs (`bcaf99b`, JM; database wiring in `c1e1b49`, SP). The `StrategistAgent` got its first real version: divergence between tone-implied and market-implied next-rate was flipped to the economically correct sign (`f327b1f`, LM) and extended with explicit narrative output (`72ec4d9`, LM). Cron-driven automation was added in three pieces: a bash wrapper layer (`8ccbc66`, `e10e03f`, SP), a 90-day documents update window (`904f796`, SP), and a final "last agent" that closes the loop (`e62cdfe`, JM). The historical scraper that backfills FOMC pages all the way to 1994 was moved into `scripts/` (`278e98c`, LM). A weighted-score bug – zero-weighted sections were polluting the average – was fixed shortly afterward (`b70b358`, JM).

2.5 Phase 5 – Productionization and Ops (May 22–27, 2026)

The static `snapshot.json` that initially powered the dashboard was replaced by a live read-only FastAPI service (`bc8cd76`, LP). The dashboard's data explorer received a legend and labels to make table semantics obvious (`32c0c38`, LP). The FOMC `FEDFUNDS` policy rate was backfilled into `macro_data` to unblock the strategist (`7a3c04c`, LM). A new `report/` directory replaced the older `literature.md` file (`39f75bd`, LM). The schema was updated to match the new analyst variants (`f53ecbf`, JM) and the README was rewritten to document all four agents end-to-end (`e1e0afd`, JM).

The final commit at the time of writing, `6cbaa7a` (LP, May 27), tightens `AGENTS.md` so that any contributor – human or AI – must treat `db/schema.sql` as authoritative and must explicitly flag missing tables or columns rather than silently introducing them. This is a direct consequence of an operational incident on the same day, where two SQLite copies of the production database had drifted: the API was reading a complete `/opt/FEDWatcher/fedwatcher.db` while all cron writers were updating an incomplete `/home/programming/FEDWatcher/fedwatcher.db`. The two databases were merged, the live API was repointed to the cron-written copy, and the redundant `/opt` tree was archived and removed. The schema-contract commit closes the door on similar drift in the future.

2.6 Tools and Workflow

Across all phases the team relied on a small, stable toolchain: **Python 3** (FastAPI, `requests`, `beautifulsoup4`, `pandas`, `sqlite3`), **SQLite** for storage, **Nginx** and `systemd` on the VPS, and **vanilla HTML/CSS/JS** (Chart.js) for the dashboard, deliberately avoiding a frontend framework. Generative-AI tools are listed in Section 7.

3 Economics and Mathematics behind the FEDWatcher

3.1 Theory

The object of interest of this project is the outcome of the next FOMC meeting, defined as the change in the upper bound of the federal funds target range relative to the current one. In order to compute the prediction, we described the possible actions of the FED through five buckets:

$$Y_{t+1} \in \mathcal{Y} = \{-50,\ -25,\ 0,\ +25,\ +50\} \text{ basis points.}$$

Then, the estimation model is a Probit that follows this specification:

$$Y_t^* = X_t\beta + \epsilon_t,$$

and the conditioning set is

$$\mathcal{X}_t = \underbrace{(\tau_t^{(1)}, \ldots, \tau_t^{(K)})}_{\text{LLM tone scores}} \cup \underbrace{(\pi_t,\ \pi_t^{\text{core}},\ u_t,\ u_t - u_t^*,\ y_t^{2y},\ r_t^{\text{ff}},\ y_t^{2y} - r_t^{\text{ff}})}_{\text{macro and rates}}.$$

Here π_t and π_t^{core} are year-on-year headline and core CPI, u_t is unemployment, u_t^* is the natural rate (FRED `NROU`) so that $u_t - u_t^*$ is the unemployment gap, y_t^{2y} is the 2-year Treasury yield, r_t^{ff} is the current Fed-funds target, and $y_t^{2y} - r_t^{\text{ff}}$ is a crude proxy for the market's pricing of forthcoming policy.

3.2 Tone Extraction

For each new document d, AnalystAgent splits the text into $K \approx 5$ sections using regex anchors trained on the structure of FOMC statements (assessment, outlook, policy stance, voting record). The LLM is then prompted with a strict JSON schema and returns, per section, an overall tone in $[-1, 1]$, an inflation assessment label (`cooling`, `stable`, `heating`), a labor-market assessment, a forward-guidance label (`dovish`, `neutral`, `hawkish`) and a confidence score. Section scores are aggregated to a single document tone by a calibrated weighted average:

$$\tau_t = \frac{\sum_k w_k \,\text{conf}_k\, s_k}{\sum_k w_k \,\text{conf}_k}, \qquad w_k \text{ from } \texttt{section\_weight\_calibration},$$

where the section weights w_k are estimated by regressing historical document tone against the realized next-meeting outcome on the training window. The dual-model analyst runs two distinct LLMs in parallel and takes a confidence-weighted average to reduce single-model variance; the resulting tone is stored in `sentiment2`/`sentiment3`, and a weighted variant in `sentiment_w`.

3.3 Nowcast Model

The baseline nowcast is an *ordered probit* over j with cutpoints $c_1 < c_2 < \cdots < c_5$:

$$Y_t^* = X_t\beta + \epsilon_t, \qquad \Pr(Y_t = j \mid \mathcal{X}_t) = \frac{\exp \mathcal{X}_t\beta_j}{1 + \sum_{k=1}^{J} \exp \mathcal{X}_t\beta_j}), \qquad c_0 = -\infty,\ c_6 = +\infty,$$

The full specification is then defined as

$$Y_t^* = \beta_1 \cdot S_t + \beta_2 \cdot \Delta CPI_t + \beta_3 \cdot Ugap_t + \epsilon_t$$

where ΔCPI_t is the variation in the Consumer Price Index, $Ugap_t$ is the unemployment gap and ϵ_t is the error term. S_t represents the tone score, extracted by a text analysis performed by an LLM from all the statements and documents published by the FED. In particular, S_t is an Exponentially Weighted Moving Average (EWMA) defined as follows:

$$\mathrm{S}_t = \alpha_t \cdot \text{tone\_score}_t + (1 - \alpha_t) \cdot \mathrm{S}_{t-1}, \quad \alpha_t = 1 - e^{-\lambda \Delta t}, \quad \text{where } \lambda = \frac{\ln(2)}{h}.$$

The first observation in our database seeds the series with alpha = 1.0 and $S_{t-1} = S_t$. After some research and literature review, we decided to set h equal to 21. We believe this is the best option for our purposes the project objectives.

3.4 Computation of Tone Implied Rate and Signal Divergence

The final tone implied rate $\hat{r}_{t+1}^{\text{tone}}$ is computed by summing to the current Fed-funds target the sum of the probabilities of each bucket j multiplied by the corresponding basis points:

$$\hat{r}_{t+1}^{\text{tone}} = r_t^{\text{ff}} + \sum_j \Pr(Y_t = j \mid \mathcal{X}_t) \cdot \text{bp}_j.$$

The StrategistAgent then computes the divergence

$$\delta_t = r_{t+1}^{\text{mkt}} - \hat{r}_{t+1}^{\text{tone}}$$

where r_{t+1}^{mkt} is the market-implied next-meeting rate from Fed-funds futures or, when unavailable, from the 1m OIS and $\hat{r}_{t+1}^{\text{tone}}$ is the tone-implied next-meeting rate (a calibrated mapping from τ_t to basis points).

4 Data Science

4.1 Data Sources

All macro series come from FRED via the public REST API: `DFEDTARU`, `DFEDTARL`, `DFF`, `FEDFUNDS`, `CPIAUCSL`, `CPILFESL`, `UNRATE`, `NROU`, `DGS2`, `SOFR`. FOMC documents are scraped from `federalreserve.gov` (statements and minutes) and from the synthetic `fakefed.ellep.it` used for parser tests. The historical backfill covers 1994–present and yields 335 documents at the time of writing.

4.2 Why SQLite

The schema (`db/schema.sql`) deliberately favors clarity over performance: every `sentiment*` table uses `document_id` as a foreign key to `documents`, and signals are reconstructable joins rather than materialized views. SQLite was chosen over Postgres because the entire dataset fits in ∼4 MB, the read pattern is dashboard-style (few writers, many readers), and a single file is trivially backupable and reproducible.

5 Sample Results and Reflection

5.1 Database Coverage

At submission the production database contains 335 FOMC documents spanning 1994-02-04 to 2026-04-29, with approximately 16 documents per year from 2015 onward (one statement + one minutes per scheduled meeting) and sparser coverage before 2000 due to gaps in the public Fed archive. Macro coverage is monthly, ∼317 observations per series, from 2000 onward. The dashboard at `https://fedwatcher.ellep.it` renders all of this live from the FastAPI service.

5.2 Tone Time Series – Qualitative Read

The aggregated tone series τ_t tracks the well-known qualitative arc of recent monetary policy: pronounced dovish dips around 2008Q4 and 2020Q1–Q2, a sustained hawkish plateau across 2022–2023, and a gradual softening starting in late 2024. Sharp single-meeting jumps in τ_t correspond, in every case we inspected, to known turning points (e.g. the December 2018 "patient" language, the March 2020 emergency cut, the November 2023 pause). This is a sanity check rather than a quantitative result: it shows that the LLM-extracted signal is not random and is broadly consistent with what an economist reading the same documents would write down.

5.3 Nowcast Performance

On the walk-forward backtest restricted to 2015–2025 (the dense-coverage era), the ordered-logit nowcast reaches approximately **XX%** top-1 accuracy and **0.YY** Brier score on six classes. A naive baseline that always predicts "hold" (the modal outcome in the sample) reaches **ZZ%**. The single LLM-tone feature contributes **Δaccuracy = ?? pp** when added on top of the macro-only model. These numbers should be read with caution: the sample is small (typically eight FOMC

meetings per year) and the model is fit on a regime that includes both ZLB and aggressive tightening, so it has not yet been stress-tested out of sample.

5.4 Divergence Signal – Case Study

We pick a representative recent meeting where δ_t flagged a tone–market gap, and walk through the document evidence the StrategistAgent retrieved, the corresponding market snapshot, and the realized outcome. The point of the case study is not predictive performance but interpretability: every quantity on the dashboard can be traced back to a specific paragraph of a specific FOMC document via the modal viewer in the document feed.

5.5 Reflection

Three things worked well: *(i)* treating `db/schema.sql` as the single source of truth (after a painful incident where two database copies drifted on the VPS, see Section 2.5); *(ii)* the AnalystAgent's structured-output prompt with a strict JSON schema, which essentially eliminated parsing errors and made downstream code trivially typed; *(iii)* an aggressively read-only API in front of SQLite – the dashboard never writes, cron never reads through the API, and the two responsibilities never conflict.

Three things did not: *(i)* early ad-hoc `ALTER TABLE` migrations created columns (`processed2`, `processed3`, `processed_w`) that were never folded back into `schema.sql`; *(ii)* the divergence sign was wrong for a week before being noticed; *(iii)* the historical Fed pre-1994 documents have inconsistent HTML and were not worth the scraping effort given their thin information content.

6 Lessons Learned

A strong project structure and scope is the key. The complexity of the project grew much more than expected. During a control meeting we realized we were facing real limitations, especially with the AnalystAgent: a single script was not enough to handle the different prompt versions, the section-splitting logic, and the various output formats we wanted to test. We decided to split the AnalystAgent into three separate agents, each writing to its own sentiment table. This decision was crucial, but this solution and the ones implemented to face other challenges would not have been possibile without the clear project structure and scope defined in the planning phase.

Schema first, code second. The biggest single source of bugs was schema drift between developer machines and the VPS. The team eventually adopted a hard rule, codified in `AGENTS.md`: `db/schema.sql` is authoritative; any table or column not declared there must not be read or written; if a feature needs a new column, the schema is updated in the same commit. Every subsequent integration error became easy to diagnose because the failing query named the missing column outright.

Multiple analysts beat one "perfect" analyst. The decision to ship *three* sentiment tables (`sentiment`, `sentiment2`, `sentiment3`) plus a weighted variant (`sentiment_w`) felt wasteful at first but turned out to be the cheapest possible ablation framework: each analyst writes to its own table with the same `document_id` foreign key, so cross-model comparisons are one SQL join away. We recommend this pattern for any LLM-in-the-loop pipeline where the prompt and segmentation are still being tuned.

A static dashboard scales further than you think. The dashboard at `https://fedwatcher.ellep.it` is hand-written HTML, CSS and one vanilla JavaScript file (`explorer.js`). It calls one or two FastAPI endpoints and renders everything client-side with Chart.js. No build step, no framework, no node_modules. Deploying a change is `scp` of a single file. For a project where the team rotates between four developers and one VPS, this was the right call.

Use AI agents for what they are good at. Generative AI was used most effectively for (a) writing the AnalystAgent prompt, (b) explaining unfamiliar pieces of the codebase to whichever team member touched them next, and (c) ops scripting (the May 27 database merge was driven by Claude Code through an SSH session). It was used least effectively for designing the schema – early LLM-suggested tables had inconsistent naming conventions and missing foreign keys, and the schema only stabilized after a human review pass.

Cron + SQLite + systemd is enough. Several alternatives were considered (Celery + Redis, Airflow, RDS) and all were rejected as over-engineering. Five cron lines and one systemd unit run the entire production pipeline. The full ops surface fits in less than 30 lines of configuration, which made the May 27 server consolidation a one-evening task rather than a sprint.

What we would do differently. (i) Write the schema and the AGENTS contract on day one, not day forty; (ii) ship the FastAPI service before the static-JSON snapshot, so the dashboard never has to migrate; (iii) integrate the divergence sign convention into a unit test, not a code review; (iv) commit more frequently with smaller diffs – some of our commits ended up bundling unrelated changes and are harder to attribute in this diary than they should be.

7 Generative-AI Acknowledgement

In accordance with the course brief, we acknowledge that generative AI was used throughout the project. The following tools were involved and the role of each is summarized for transparency.

Anthropic Claude (Claude Code, Opus 4.6 and 4.7)
Used as the primary pair-programming agent: code edits, refactors, AGENTS.md drafting, the May 27 server consolidation (database merge, systemd repointing, archive download), and the draft of this very document. Invoked through the Claude Code CLI, not the chat UI, so that the full conversation, tool calls and diffs are reviewable.

OpenAI GPT-4 / GPT-4o (via OpenRouter)
Used as the secondary LLM in the dual-model AnalystAgent and for spot checks of the Claude-produced JSON output. Access goes through OpenRouter so that the API key is provider-agnostic.

GitHub Copilot
Used opportunistically in-editor for line-level completions, mostly in `explorer.js` and the FRED ingestion scripts.

Foreign code and citations. The project depends on standard open-source libraries: `fastapi`, `uvicorn`, `requests`, `beautifulsoup4`, `pandas`, `numpy`, `scikit-learn`, and `Chart.js`. All are listed in `requirements.txt` or loaded from a CDN in `fedwatcher/index.html`. No code was copied verbatim from external tutorials or Q&A sites; where an approach was inspired by an online reference (e.g. the FOMC regex anchors), the comment block in the relevant file names the source. The FRED data is public-domain via the Federal Reserve Bank of St. Louis API; the Fed documents are public-domain.

Repository and reproducibility. The full source code, commit history, issues and pull requests are available at https://github.com/LeoPalanca/FEDWatcher. Installation and deployment steps are in `README.md`; the cron schedule and systemd unit are reproduced verbatim in `deploy/`. Anyone with the GitHub URL can recreate the entire pipeline on a fresh VPS in under an hour.